

SLE-4: Architecture Decision Record (ADR)

Course: 02AML204 – Introduction to Artificial Intelligence

PRN: 25UAM116

Name: Madhur Pravin Bhandari

Division: B

Date: 8 October 2026

GitHub / Portfolio Link: https://github.com/madhur-184/SLE-4_IAI

PART A: Architecture Decision Record (ADR)

Title: Use of Breadth-First Search (with an Explored Set) as the Solver for the 8-Puzzle, instead of Depth-Limited DFS

Status: Accepted

Date: 8 October 2026

PRN & Name: 25UAM116 – Madhur Pravin Bhandari

1. Context

My system is an 8-Puzzle solver and profiler (SLE-1). It takes a scrambled 3×3 board and returns a sequence of blank-tile moves that reaches the goal state. I had to choose which uninformed search strategy should be the main solver. The choice matters because it decides whether the answer is the shortest possible and how much work and memory the search needs. In SLE-2 I profiled BFS and depth-limited DFS (limit 30) on 4, 12 and 22-move scrambles, and in SLE-3 I drew the C4 model that keeps the move logic common to both.

2. Decision

I chose Breadth-First Search, using a FIFO queue as the Frontier and an Explored Set to avoid repeated states, as the main solver for the 8-Puzzle. DFS stays in the project only as a comparison baseline. Both share the same Problem Definition container (get_neighbors) and return the same (path_length, nodes_expanded) pair.

3. Alternatives Considered

- **Alternative 1 – Depth-limited DFS (limit 30):** profiled in SLE-2. Low memory, but not optimal and very sensitive to neighbour order.
- **Alternative 2 – A* with Manhattan-distance heuristic:** would expand far fewer nodes, but needs a heuristic module that I have not built yet.
- **Alternative 3 – Iterative-Deepening DFS (IDDFS):** optimal with $O(d)$ memory, but it repeats work at every depth level, so it is slower than BFS on this small state space.

4. Rationale / Justification

The decision is based mainly on my SLE-2 measurements (5 runs per case, time.perf_counter, manual node counter):

Test case	Algo	Time (ms)	Nodes expanded	Path length
4-move scramble	BFS	0.0364	24	4
4-move scramble	DFS (limit 30)	43.97	42,235	4
12-move scramble	BFS	1.664	1,128	12
12-move scramble	DFS (limit 30)	35.42	34,661	30
22-move scramble	BFS	53.91	31,523	18
22-move scramble	DFS (limit 30)	13.02	13,846	30

- **Optimality:** BFS returned 4, 12 and 18 moves, which are the shortest paths. DFS returned 30 moves on two of the three puzzles, because it dives to its depth limit first. On the 22-move puzzle that is 66% longer than the optimal 18.
- **Efficiency on easy and medium puzzles:** on the 4-move case BFS needed 24 nodes versus 42,235 for DFS. On the 12-move case it needed 1,128 versus 34,661. Averaged over the 3 cases, BFS expanded 10,892 nodes against 30,247 (about 2.8× fewer), and its average time was 18.54 ms against 30.80 ms.
- **Honest exception:** on the 22-move puzzle DFS was faster (13.02 ms vs 53.91 ms) and expanded fewer nodes. That was luck, since its first branch happened to lead towards the goal, and the path it returned was not the shortest. DFS cost depends on neighbour order, so it is unpredictable.
- **Bounded memory here:** the 8-Puzzle has only $9!/2 = 181,440$ reachable states, so BFS's $O(b^d)$ memory stays manageable. The Explored Set (SLE-3, Level 3) guarantees no state is expanded twice.
- **Fits my architecture:** only the Frontier changes (queue for BFS, stack for DFS). Because both use the same Problem Definition container, the SLE-2 comparison was fair and the choice can be changed later without touching the other containers.

5. Consequences

Positive:

- Always gives the shortest solution, so results are predictable and easy to verify.
- Simple to implement, test and explain with the C4 diagrams.
- The modular design lets me add a Heuristic Module (7th container) and move to A* without a rewrite.

Negative / Trade-offs:

- Memory grows as $O(b^d)$; this would fail on a 15-puzzle, which has about 10^{13} states.
- Can be slower than DFS on a lucky case (22-move puzzle) and expands many nodes on deep scrambles (31,523 nodes at depth 18), because it explores blindly with no heuristic.

PART B: Short Summary of My SLE Journey

In SLE-1 I built a Simple AL Bot. In SLE-2 I built an 8-Puzzle solver with BFS and depth-limited DFS and profiled both with py-spy and time.perf_counter on 4, 12 and 22-move scrambles. BFS was optimal (4/12/18 moves) and usually cheaper in nodes, while DFS returned 30-move paths. In SLE-3 I drew the four C4 levels, splitting the system into 6 containers and zooming into the Search Engine, and kept get_neighbors() shared so the comparison stayed fair. In SLE-4 I turned these findings into one decision: use BFS as the main solver, and treat A* with a heuristic as the next step for larger puzzles.

PART C: AI Contribution Note

- **AI tools used:** Claude (Anthropic).
- **What AI helped with:** drafting the ADR layout from the SLE-4 guideline, summarising my SLE-2 and SLE-3 reports into the ADR sections, and formatting this Word file.
- **What I did myself:** built and profiled the BFS/DFS solver, collected the SLE-2 numbers, designed the C4 model, chose which decision to record, and checked that every number and component name here matches my own work. I will explain and defend the decision in the viva.